Model Augmentation for a Dual-Gantry High-Precision Motion System

Dirk van den Berg
Control Systems Group, Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands

Abstract

I. Introduction

ASMPT develops semiconductor assembly and packaging equipment, including high-precision motion systems for advanced packaging, which combines multiple semiconductor functions within a single package. Such technologies place strict demands on alignment accuracy, yield, and manufacturing cost [1], [2].

This paper focuses on a dual-gantry motion system in which two parallel linear actuators support a common beam carrying the payload. The shared beam couples the actuator dynamics, so motion of one actuator influences the other, and the system exhibits position-dependent behaviour [3], [4] and flexible dynamics [3]. In such systems, settling behaviour is a key performance metric. After each motion, the system must wait for residual oscillations to decay before the next placement can begin [5], so settling time directly affects throughput. Furthermore, cross-coupling means that the motion of one gantry can disturb the other, making accurate prediction relevant not only for settling but also for tracking behaviour during simultaneous motion.

Predicting this behaviour therefore requires an accurate representation of the underlying plant dynamics. Such a model can be used as a digital twin, for example to evaluate a changed controller before it is tested on the machine. Digital twins can support offline simulation of semiconductor manufacturing equipment and help optimize operating parameters [6]. Such models are also valuable for control design and for process and root cause analysis. This requires a model with physically meaningful parameters, because a change in machine behaviour can then be traced back to a physical cause, such as a change in stiffness or damping.

White-box models are attractive because they reflect the physics of the system, which makes them interpretable. First-principles (FP) models are a specific type of white-box model, derived from physical laws and described by quantities such as mass, stiffness, and damping. However, purely physics-based models do not always capture all behaviour needed for accurate prediction across the operating range [7]. More accurate FP models can be developed, but this is labour-intensive [8].

Purely data-driven (black-box) models such as neural networks or Gaussian processes can describe measured behaviour well, but they give less physical insight and are often harder to trust at operating conditions not covered by the training data [7]. Typically these models also require large amounts of data to train [7], [9]. In addition, these models need to learn behaviour that is already known from first principles, such as the rigid-body dynamics [8]. Physics-informed approaches, including physics-informed neural networks (PINNs), enforce physical laws as constraints during training [9], [10]. However, these methods require the physics equations to be known and still rely on a black-box model for the entire system [8]. For the plant model of the dual-gantry system, both predictive accuracy and physical interpretability are required. Physical interpretability means that the estimated physical parameters are unique [11]. However, neither a purely physics-based nor a purely data-driven model is sufficient on its own [7]. Grey-box modeling is the logical choice, because it is able to combine a physically meaningful baseline model needed for interpretation with data-driven flexibility needed for accurate prediction [12]. The following grey-box approaches appear in practice: parameter re-estimation within the physics model structure, static residual correction on the model output, and dynamic augmentation that extends the baseline with learned states [7], [8]. Model augmentation combines a physics-based baseline model with a learned component, so that the learned component only needs to capture the dynamics the physics-based model cannot describe [8]. However, related work on an ASMPT wire bonder shows that preserving physical interpretability during model updating remains an open challenge [13].

Learning-based hybrid modeling has already been demonstrated on real industrial systems [14]. However, a key gap remains: there is no grey-box modeling framework for the dual-gantry system that simultaneously addresses coupled multi-input multi-output (MIMO) dynamics, position-dependent behaviour, closed-loop data, and preserves physical interpretability. Model augmentation using a linear fractional representation (LFR) has already been demonstrated, but not for position-dependent baselines or closed-loop data [8]. The related work at ASMPT uses closed-loop data, but keeps the physical parameters fixed during training [13]. Orthogonal-by-construction augmentation is able to preserve the physical meaning of the baseline parameters, but has so far only been formulated for discrete-time input-output models [11]. This paper therefore aims to develop and evaluate such a framework in simulation, so that the plant model becomes more accurate across operating points and motion profiles while remaining physically meaningful.

This paper addresses the following research question: *For the dual-gantry system, how should a physics-based plant model be augmented to capture unmodelled dynamics, improving prediction accuracy across operating points and motion profiles while preserving the physical interpretability of its parameters?* To answer this research question, the following four aspects are investigated:

1) In what form should the baseline model be represented for subsequent augmentation, and does a position-dependent (LPV) formulation improve prediction compared to position-independent (LTI) alternatives?
2) What augmentation structure can effectively capture dynamics not modeled in the physics-based baseline model?
3) How can parameter interpretability be maintained during data-driven augmentation, compared to unconstrained approaches?
4) How well does the augmented model generalize to (i) held-out operating points and (ii) unseen motion profiles, and how does its prediction accuracy compare to a black-box state-space model?

II. Dual-Drive Gantry System and Physics Baseline

A. System and first-principles baseline

The physics baseline of the gantry system used in this paper is the lumped-parameter model of the the dual-drive gantry of García-Herreros et al. [3]. The gantry has two actuators that are connecected with a cross-arm and a third actuator that moves the payload along this arm (Fig. 1). Define the actuator forces $u_{\text{act}} = [F_{X1}\ F_{X2}\ F_Y]^\top$, the measured positions $q_{\text{act}} = [X_1\ X_2\ Y]^\top$, and the generalized coordinates $q = [X\ \Theta\ Y]^\top$. Let L_b denote the cross-arm length between the two flexible joints. The coordinate and force transformations are

$$q = P^{-\top} q_{\text{act}}, \qquad u = P u_{\text{act}}, \qquad P = \begin{bmatrix} 1 & 1 & 0 \\ L_b/2 & -L_b/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

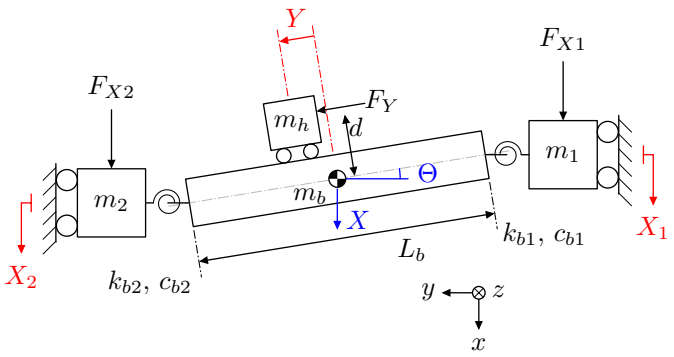


Fig. 1:

where u denotes the generalized force. The generalized coordinates are defined as $X = (X_1+X_2)/2$ and $\Theta = (X_1-X_2)/L_b$, while Y is unchanged. These coordinates avoid [3]. Moving the payload position Y changes the mass distribution of the gantry and therefore its inertia. This position dependence is retained by representing the baseline in linear parameter-varying (LPV) form, with the measured payload position Y as scheduling variable [15].

Following García-Herreros et al. [3], the baseline uses $\cos\Theta \approx 1$ and $\sin\Theta \approx 0$ and assumes that the Coriolis and centripetal force term is negligible compared with the inertia, damping, and stiffness forces. Under these assumptions, the baseline equations of motion are

$$M(Y)\ddot{q}(t) + C\dot{q}(t) + Kq(t) = P u_{\text{act}}(t), \tag{2}$$

with

$$M(Y) = \begin{bmatrix} m_1+m_2+m_b+m_h & \frac{L_b}{2}(m_1-m_2)-m_hY & 0 \\ \frac{L_b}{2}(m_1-m_2)-m_hY & J_b+J_h+\frac{L_b^2}{4}(m_1+m_2) & -m_hd \\ & +m_h(d^2+Y^2) & \\ 0 & -m_hd & m_h \end{bmatrix}, \tag{3}$$

$$C = \begin{bmatrix} c_{g1}+c_{g2} & \frac{L_b}{2}(c_{g1}-c_{g2}) & 0 \\ \frac{L_b}{2}(c_{g1}-c_{g2}) & c_{b1}+c_{b2}+\frac{L_b^2}{4}(c_{g1}+c_{g2}) & 0 \\ 0 & 0 & c_y \end{bmatrix}, \tag{4}$$

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & k_{b1}+k_{b2} & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

The masses, flexible-joint stiffness and damping coefficients, and are shown in Fig. 1. Here, J_b and J_h denote the yaw inertias, while c_{g1}, c_{g2}, and c_y denote the actuator damping coefficients. The nominal values are taken from [3, Table I] and listed in Appendix A-C To prevent the definition of q from being parameter-dependent, the cross-arm length L_b is fixed at the nominal value listed in Appendix A-C rather than treated as a free model parameter.

B. Self-scheduled LPV-LFR representation

The measured payload position Y is both a state and the scheduling variable, making the baseline a self-scheduled LPV model . Solving (2) for $\ddot{q}$ requires $M(Y)^{-1}$, so the model is rationally dependent on Y. An Linear Fractional Representation (LFR) is introduced to separate this rational scheduling dependence. We move the dependence out of the LTI part G and into a Delta-block $\Delta(Y)$, with the scheduling being connected in feedback with G (Fig. 2) [8], [16].

To separate the scheduling variable Y, deconstruct the inertia matrix (3) into its Y dependent components,

$$M(Y) = M_0 + YM_1 + Y^2M_2. \tag{5}$$

M_0, M_1 and M_2 are given in Appendix A-A. Substituting (5) into (2) gives

$$M_0\ddot{q} + M_1(Y\ddot{q}) + M_2(Y^2\ddot{q}) = u - C\dot{q} - Kq. \tag{6}$$

Here Y only enters through $Y\ddot{q}$ and $Y^2\ddot{q}$, with $Y^2\ddot{q} = Y(Y\ddot{q})$. This lets us apply the same scheduling variable twice instead of introducing a second scheduling variable Y^2,

$$\ddot{q} \xrightarrow{Y} a_1 \xrightarrow{Y} a_2, \tag{7}$$

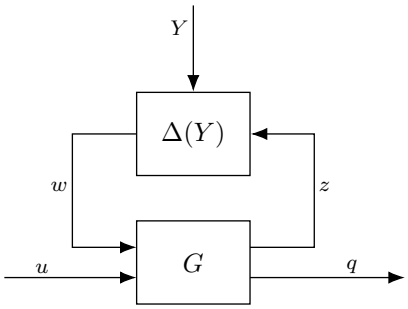


Fig. 2:

with $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$.

We treat a_1 and a_2 as independent signals (6) becomes

$$M_0\ddot{q} + M_1 a_1 + M_2 a_2 = u - C\dot{q} - Kq, \tag{8}$$

which contains no Y. The latent variables are $z = \mathrm{col}(\ddot{q}, a_1)$ and $w = \mathrm{col}(a_1, a_2)$, with the Delta-block mapping z to w.

$$w = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} Y\ddot{q} \\ Ya_1 \end{bmatrix} = \underbrace{\begin{bmatrix} YI_3 & 0 \\ 0 & YI_3 \end{bmatrix}}_{\Delta(Y)=YI_6} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \Delta(Y)z. \tag{9}$$

We do not claim this realization to be minimal . We can write the LTI part G of the LFR in the continous-time form, [16, Eq. (6)],

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = G \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}, \tag{10}$$

where $\tilde{x} = \mathrm{col}(q, \dot{q})$ is the state and q the output. The first block row is the state equation, with $\ddot{q}$ obtained from (8) by multiplying with M_0^{-1}. For the blocks of G Appendix A-A. Closing the algebraic loop with (9) results in $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$, which turns (8) back into (2). The LFR is therefore an exact realization of the baseline. The LFR is well posed if the algebraic loop has a unique solution for every Y [16, Def. 1]. For the gantry realization we show that this conditions holds when $M(Y)$ is nonsingular (Appendix A-A). For positive masses and inertias, $M(Y) \succ 0$ for every $Y \in \mathbb{R}$ (Appendix A-D). Therefore the LFR is well posed over the whole Y-operating range.

C. Physical parameters and identifiable combinations

The fourteen parameters θ of Table III cannot all be identified from input-output data. The positions X_1, X_2, Y are measured, which gives q through (1), and the forces u are known in newtons. Two parameter sets θ and θ' that produce the same motion for every input force satisfy

$$\big(M(Y) - M'(Y)\big)\ddot{q} + (C - C')\dot{q} + (K - K')q = 0$$

With unconstrained initial conditions and full actuation, any acceleration $\ddot{q}$ can be reached at any state within the operating range. Therefore the identity must hold for all these states and accelerations, this implies that $K = K'$, $C = C'$ and $M(Y) = M'(Y)$. Since $M(Y)$ is quadratic in Y, this also implies that $M_0 = M_0'$, $M_1 = M_1'$ and $M_2 = M_2'$. If these matrices are equal, the input-output behaviour is also the same. Therefore, the part of θ that can be identified is determined by the parameter combinations in M_0, M_1, M_2, C and K. If we ignore the symmetric entries, (3) and (4) contain twelve nonzero entries. Three of these, $M_{0,33}$, $-M_{1,12}$ and $M_{2,22}$, are equal to m_h. This leaves us with ten distinct functions of θ. The Jacobian of these ten functions with respect to $(m_h, d, m_1, m_b, J_b, c_{g1}, c_{g2}, c_{b1}, c_y, k_{b1})$ is block triangular and has a determinant of $m_h L_b^2/2 \neq 0$, meaning that the then functions are independent. Since the matrices only depend on these ten functions, exactly ten independent combinations of θ can be identified [17, Def. 7].

We define these ten combinations as

$$\theta_{\mathrm{base}} = [k_{b,\Sigma}, c_{g1}, c_{g2}, c_y, c_{b,\Sigma}, m_h, m_\Sigma, m_\Delta, J_{\mathrm{eff}}, d]^\top, \tag{11}$$

with $k_{b,\Sigma} = k_{b1} + k_{b2}$, $c_{b,\Sigma} = c_{b1} + c_{b2}$, $m_\Sigma = m_1 + m_2 + m_b$, $m_\Delta = m_1 - m_2$ and $J_{\mathrm{eff}} = J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2)$. This follows the idea of base parameters used in robot dynamics [18]. For $m_h > 0$, θ_{base} can be uniquely determined from the ten entries (Appendix A-B). We therefore estimate θ_{base} instead of θ, presented in the results .

Positive θ_{base} does not imply $M(Y) \succ 0$. For $m_h, m_\Sigma, J_{\mathrm{eff}} > 0$, $M(Y) \succ 0$ for every Y if and only if

$$m_\Sigma J_{\mathrm{eff}} > \tfrac{L_b^2}{4}\, m_\Delta^2 \tag{12}$$

(Appendix A-D). To ensure that well-posedness and (12) hold during training, we use a logarithmic parameterization for the nine positive combinations and a $\tanh$-bounded m_Δ (Section III-D).

III. Dynamic Augmentation of the LPV-LFR Baseline

The dynamics of the system that are not captured by the baseline model defined in Section II-B will be identified from input-output data measured in closed-loop. The baseline model will be augmented to represent these missing dynamics. A correction to the states of the baseline model alone is not sufficient, so additional states are introduced. We make use of a neural network to model both the changes for the baseline states and the additional states.

The model is trained using short windows of the closed-loop data, where each window is simulated with the known controller. The initial states are estimated by an encoder, including the additional states. During training, the physical parameters θ_{base} are also estimated, using the combinations defined in Section II-C.

A. Augmented model

To represent additional dynamics such as a flexible mode, refitting θ_{base} or correcting the baseline states with a static augmentation is not sufficient. Both these methods the original states of the baseline, while missing dynamics require states the baseline does not include [8, Sec. 2]. A dynamic augmentation therefore adds states $\bar{x}$ to model these missing dynamics [8, Sec. 2]. The augmented model updates these states using the

full state and input [8, Sec. 2, Table 1].

The LFR-based augmentation structure of Hoekstra et al. is used, an interconnection matrix connects a baseline block and a learning block [8, Sec. 3.1]. The learning components are a neural network consisting of f_{aug} and g_{aug}. Similar to the LFR in Section II-B (Fig. 2), a constant matrix connects the two blocks through latent variables. The self-scheduled LFR The baseline block for this is the self-scheduled LFR of Section II-B, this is a nonlinear map of the state and the input [19, Sec. 1.1]. The augmented model is thus an LFR whose baseline block is itself an LFR in Y.

The dynamic-parallel augmentation structure is chosen, because the orthogonal-by-construction parametrisation that Section IV builds upon is defined for the additive structure [11].

The output map is not augmented (no h_{aug}), because (1) gives the exact relation between the states and the measured outputs. This is also done to ensure that q still represents the physical positions. With a learned correction to the output map the states could stop representing the actual positions. Kessels also leaves the output map unchanged when it is already accurate [13, Remark 5.3].

The model is defined in discrite time with the augmentation added to the state update of the baseline [8, Sec. 2, Table 1]. This gives us

$$\begin{aligned}\tilde{x}_{k+1} &= f_{\text{base}}(\theta_{\text{base}}, \tilde{x}_k, u_{\text{act},k}) + f_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \bar{x}_{k+1} &= g_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \hat{y}_k &= P^\top q_k,\end{aligned} \tag{13}$$

where $\tilde{x}_k = \text{col}(q_k, \dot{q}_k) \in \mathbb{R}^6$ is the physical state of (10), $\bar{x}_k \in \mathbb{R}^{n_{\bar{x}}}$ the added state, $\hat{x}_k = \text{col}(\tilde{x}_k, \bar{x}_k)$, $u_{\text{act},k} \in \mathbb{R}^3$ the actuator forces, $\hat{y}_k \in \mathbb{R}^3$ the predicted actuator positions, and $y_k = q_{\text{act},k}$ the measured positions, $\theta_{\text{base}} \in \mathbb{R}^{10}$ the combinations (11), and $\theta_{\text{aug}} \in \mathbb{R}^{n_{\text{aug}}}$ the network parameters. The added states are updated by g_{aug} without any predetermined physical meaning, in contrast to the baseline states $\tilde{x}$ for which the positions are measured.

The baseline is evaluated over one sample period T_s. Writing (2) in state-space form gives

$$\dot{\tilde{x}} = F(\tilde{x}, u) = \begin{bmatrix} \dot{q} \\ M(Y)^{-1}\big(u - C\dot{q} - Kq\big) \end{bmatrix}, \tag{14}$$

where $u = Pu_{\text{act}}$ is the generalized force (1), M, C and K depend on θ_{base} through (11), and $M(Y)^{-1}$ is the closed loop of the LFR of Section II-B. f_{base} is calculated using one fourth-order Runge-Kutta (RK4) step of F over T_s, with the input kept constant during the step. For each of the four RK4 steps $M(Y)$ is evaluated using the value of Y from its intermediate state.

The inputs to the neural network are normalised to zero mean and unit variance, which has been shown to improve the estimation of models containing neural networks [8, Sec. 5.3]. The mean and standard deviations are computed from the training data. For the baseline states the measured positions are converted to their generalized positions, for which the velocities can be estimated from their finite difference by dividing with T_s. This ensures that the positions and outputs are scaled with the same measurements: keeping the scales consistent. This also makes sure that all models independent of its initial parameters use the same scale,

$$\tilde{x}^{\text{n}} = T_x(\tilde{x} - \mu_x), \quad u^{\text{n}} = T_u(u_{\text{act}} - \mu_u), \quad y^{\text{n}} = T_y(y - \mu_y), \tag{15}$$

where $\mu_x \in \mathbb{R}^6$ and $\mu_u, \mu_y \in \mathbb{R}^3$ are the channel means, and $T_i = \text{diag}(\sigma_i)^{-1}$ for $i \in x, u, y$, with σ_i the corresponding standard deviations. The additional states $\bar{x}$ are not normalised, so $\hat{x}^{\text{n}} = \text{col}(\tilde{x}^{\text{n}}, \bar{x})$.

After normalisation the model equations become

$$\begin{aligned} f^{\text{n}}_{\text{base}}(\theta_{\text{base}}, \tilde{x}^{\text{n}}, u^{\text{n}}) &= T_x\big(f_{\text{base}}(\theta_{\text{base}}, \tilde{x}, u_{\text{act}}) - \mu_x\big),\\ f^{\text{n}}_{\text{aug}} = T_x f_{\text{aug}}, &\qquad g^{\text{n}}_{\text{aug}} = g_{\text{aug}},\\ \hat{y}^{\text{n}}_k = C^{\text{n}}\tilde{x}^{\text{n}}_k, &\qquad C^{\text{n}} = T_y P^\top [I_3 \;\; 0]\, T_x^{-1}. \end{aligned} \tag{16}$$

where $\tilde{x}$ and u_{act} follow from $\tilde{x}^{\text{n}}$ and u^{n} by inverting (15), so RK4 uses the states and inputs in physical units. The correction represents a change in the states itself and not the full state itself, and therefore we only scale it. The output map remains linear because the mean of the positions is calculated from the output mean as $P^{-\top}\mu_y$.

A multilayer perceptron ϕ_{aug} with tanh activation functions for the hidden layers and a linear output layer is used for both $f^{\text{n}}_{\text{aug}}$ and $g^{\text{n}}_{\text{aug}}$:

$$\text{col}(f^{\text{n}}_{\text{aug}}, g^{\text{n}}_{\text{aug}}) = \phi_{\text{aug}}(\theta_{\text{aug}}, \hat{x}^{\text{n}}_k, u^{\text{n}}_k).$$

The inputs to the neural network are the $6 + n_{\bar{x}}$ states and the three actuator forces. The first six outputs $f^{\text{n}}_{\text{aug}}$ update the six physical states and the remaining $n_{\bar{x}}$ outputs $g^{\text{n}}_{\text{aug}}$ and equals the next additional states $\bar{x}_{k+1}$ (Fig. 3). These corrections act on all six physical states, the positions included, as in the dynamic-parallel structure [8, Table 1]. The positions are also corrected because within one sample period an unmodelled force would change both the velocity and position, correcting only for the velocities would not be able to capture this. Both f_{aug} and g_{aug} are given the payload position Y through $\tilde{x}_k$, enabling the neural network to learn model position-dependent effects.

Randomly initializing the learning components can give an unstable and bad optimization problem, to resolve this training starts from the baseline [8, Sec. 5.4]. At the start of training f_{aug} is set to zero, resulting in the augmented model being equal to the baseline. Because we make use of a feedforward neural network this only requires us to zero the output layer.

The gantry baseline model has two free axes, X and Y, without stiffness in (4), which are also affected by the learned correction. The discretized baseline model has a pole at $z = 1$ on each of these stiffness free axes, making it marginally stable. Therefore these axes act as integrators: a consant error in the correction keeps accumalating in the position. The alternative to only correct the velocities would not prevent the integrator issue, because an error in the velocity also accumulates in the position. The feedback controller is able to correct this drift, which is why we have chosen to simulate the model in closed-loop (Section III-C).

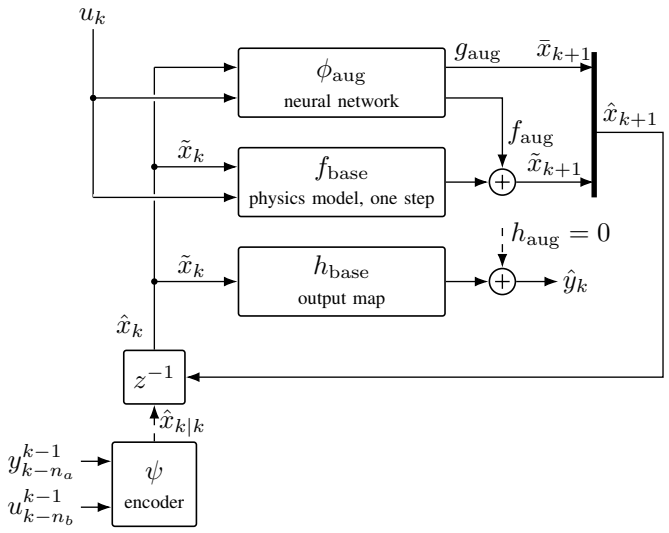


Fig. 3: Augmented model. The physics step f_{base} and the network ϕ_{aug} act in parallel on $\hat{x}_k$. ϕ_{aug} supplies f_{aug} and g_{aug}, and the output map is not learned. The encoder ψ sets $\hat{x}$ at each window start. During training, the input is the closed-loop input of Section III-C.

B. Encoder and initial state

Only the positions of the state at the start τ of a training window are measured. Following Beintema et al., an encoder estimates this state from the input and output history [20]. Simulating short windows from encoded states keeps the cost of a gradient step independent of the record length and stabilises the optimisation [20, Secs. 2, 3]. Unlike the criterion of Hoekstra et al., whose encoder reads the samples up to $\tau - 1$ [8, Eq. (22e)], the history includes sample τ, as the reconstructability map used below requires [21, Eqs. (11) to (17)]:

$$\begin{aligned} \hat{x}^{\mathrm{n}}_{\tau|\tau} &= \psi(\theta_{\mathrm{enc}}, \chi_\tau), \\ \chi_\tau &= \mathrm{col}\big(y^{\mathrm{n}}_{\tau-n}, \ldots, y^{\mathrm{n}}_{\tau},\, u^{\mathrm{n}}_{\tau-n}, \ldots, u^{\mathrm{n}}_{\tau}\big), \end{aligned} \tag{17}$$

where n is the encoder history, $\chi_\tau \in \mathbb{R}^{6(n+1)}$ the recorded window and $\theta_{\mathrm{enc}} \in \mathbb{R}^{n_{\mathrm{enc}}}$ the encoder parameters. The encoder thus estimates all $6 + n_{\bar{x}}$ states, the measured positions included.

The encoder is a linear map with a nonlinear correction [21, Eq. (8)]. Its added-state rows W^a have no baseline counterpart. The linear part acts on the scaled but uncentred window, for which the map (20) below is built:

$$\psi(\theta_{\mathrm{enc}}, \chi_\tau) = \begin{bmatrix} W^b \\ W^a \end{bmatrix} (\chi_\tau + \chi_0) - \begin{bmatrix} T_x \mu_x \\ 0 \end{bmatrix} + \tilde{\psi}(\chi_\tau), \tag{18}$$

where $W^b \in \mathbb{R}^{6\times 6(n+1)}$, $W^a \in \mathbb{R}^{n_{\bar{x}}\times 6(n+1)}$, χ_0 restores the means that (15) removes, $\tilde{\psi}$ is a neural network, and θ_{enc} collects W^b, W^a and the parameters of $\tilde{\psi}$. We write ψ_a for the last $n_{\bar{x}}$ rows of ψ and θ^a_{enc} for W^a with the added-state rows of the output layer of ψ.

The physical rows start from the baseline, through the model-based initialisation of Hoekstra et al. [21, Sec. 3.1]. Its map needs a linear model, so as in [21, Eqs. (30) to (32)] we linearise the baseline at an equilibrium. We take rest at $Y = 0$, which is an equilibrium for zero input because K in (4) vanishes on X and Y, and the position at which the controller of the records is designed (Section V). Frozen at $Y = 0$, (2) is linear, so the linearisation is the baseline itself at that position and approximates it only as Y moves. Discretising by zero-order hold over T_s and scaling as (15) without the means gives

$$\begin{aligned} &A_d = T_x e^{A_c T_s} T_x^{-1}, \quad B_d = T_x \int_0^{T_s} e^{A_c s}\,\mathrm{d}s\, B_c T_u^{-1}, \quad C_d = T_y C_c T_x^{-1}, \\ &A_c = \begin{bmatrix} 0 & I_3 \\ -M_0^{-1}K & -M_0^{-1}C \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 \\ M_0^{-1}P \end{bmatrix}, \quad C_c = \begin{bmatrix} P^\top & 0 \end{bmatrix}, \end{aligned} \tag{19}$$

with $M_0 = M(0)$ from (5), evaluated at the nominal parameters. Stacking the outputs and inputs of the window as Y_τ and U_τ, the output and input parts of $\chi_\tau + \chi_0$ with the oldest sample first, gives $Y_\tau = O_n \tilde{x}_{\tau-n} + \Gamma_n U_\tau$ and $\tilde{x}_\tau = A_d^n \tilde{x}_{\tau-n} + r_n U_\tau$. Eliminating $\tilde{x}_{\tau-n}$ gives the reconstructability map [21, Eqs. (15) to (17)]

$$\begin{aligned} \tilde{x}_\tau &= A_d^n O_n^+ \big(Y_\tau - \Gamma_n U_\tau\big) + r_n U_\tau, \\ W^b &= \begin{bmatrix} A_d^n O_n^+ & r_n - A_d^n O_n^+ \Gamma_n \end{bmatrix}, \end{aligned} \tag{20}$$

where $O_n = \mathrm{col}(C_d, C_d A_d, \ldots, C_d A_d^n) \in \mathbb{R}^{3(n+1)\times 6}$, Γ_n is block lower triangular with block (i, j) equal to $C_d A_d^{i-j-1} B_d$ for $i > j$ and zero otherwise, $r_n = [A_d^{n-1} B_d \ \cdots \ B_d \ 0]$, and $^+$ is the Moore-Penrose pseudoinverse. The paper assumes a stable baseline [21, Sec. 2]. Ours is marginally stable, but the measured positions make (A_d, C_d) observable, so O_n has full column rank. The map is derived without noise; with white Gaussian measurement noise it is the maximum-likelihood estimate of the state [21, Sec. 3.1]. The paper also gives a version for process noise, which needs a known noise gain [21, Eqs. (18) to (29)]. The noise in our records enters at the position measurement (Section V) and no noise gain is known, so we use (20). Our measurement noise is coloured, so the map is a starting value for W^b, not an optimal estimator. It leaves co-estimation of the baseline parameters out of scope [21, footnote 1], whereas here W^b is trained jointly with them.

Hoekstra et al. instead fit the baseline encoder to states simulated with the baseline [8, Eq. (30)], the data-based alternative of [21, Sec. 3.2]. On the gantry, that simulation is driven by the recorded input and drifts on the free axes X and Y (Section III-C), so it would fit W^b to wrong states. The map (20) needs no simulation and negligible computation [21, Sec. 4.4]. It gives better starting values and faster convergence than a random encoder, at comparable final accuracy [21, Sec. 4.4].

C. Closed-loop simulation

The records are measured under feedback, and the model must predict the machine under feedback: its response to a reference, a feedforward and a known controller. We therefore simulate each training window in closed loop with that controller, rather than driving the model with the recorded input as in [8, Eq. (22)]. Kessels trains augmented models in

this way [13, Eqs. (5.12), (5.13)] and reports that identifying the open-loop model from his closed-loop data failed, which he attributes to noise-induced bias [13, Remark 5.6]; our records are noisy as well (Section V). For linear systems, fitting the closed loop from the reference with a known controller is the indirect approach to closed-loop identification, in which an output-error criterion remains consistent because the reference is uncorrelated with the noise [22]. The free axes give a second reason. X and Y have no stiffness, so the open-loop gantry is marginally stable: a difference in initial velocity leaves a permanent position offset. The consistency argument of the identification method [8, Sec. 5.5] rests on that of SUBNET, which assumes an incrementally exponentially output stable data-generating system [20, Cond. 1]; the open-loop gantry violates this condition, whereas the loop with the stabilising controller can satisfy it. For our nonlinear model, both arguments motivate the closed loop but do not prove consistency.

The controller of each record is known, linear and time invariant, and maps the position error in actuator coordinates to actuator forces. The machine and the model each close this controller around their own output, with controller states x^{fb} and $\hat{x}^{\mathrm{fb}}$, and both loops are driven by the reference r and feedforward u^{ff} of the record (Fig. 4):

$$\begin{aligned} x^{\mathrm{fb}}_{k+1} &= A_{\mathrm{fb}} x^{\mathrm{fb}}_k + B_{\mathrm{fb}}(r_k - y_k), \\ u_{\mathrm{act},k} &= C_{\mathrm{fb}} x^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - y_k) + u^{\mathrm{ff}}_k, \\ \hat{x}^{\mathrm{fb}}_{k+1} &= A_{\mathrm{fb}} \hat{x}^{\mathrm{fb}}_k + B_{\mathrm{fb}}(r_k - \hat{y}_k), \\ \hat{u}_{\mathrm{act},k} &= C_{\mathrm{fb}} \hat{x}^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - \hat{y}_k) + u^{\mathrm{ff}}_k, \end{aligned} \tag{21}$$

where $(A_{\mathrm{fb}}, B_{\mathrm{fb}}, C_{\mathrm{fb}}, D_{\mathrm{fb}})$ are the state-space matrices of the controller discretised over T_s, r_k and u^{ff}_k are in actuator coordinates, and $\hat{u}_{\mathrm{act},k}$ is the input the model receives in place of the recorded $u_{\mathrm{act},k}$. Subtracting the machine's lines from the model's, with $x^c = \hat{x}^{\mathrm{fb}} - x^{\mathrm{fb}}$, removes r and u^{ff}:

$$\begin{aligned} x^c_{k+1} &= A_{\mathrm{fb}} x^c_k + B_{\mathrm{fb}}(y_k - \hat{y}_k), \qquad x^c_\tau = 0, \\ \hat{u}_{\mathrm{act},k} &= u_{\mathrm{act},k} + C_{\mathrm{fb}} x^c_k + D_{\mathrm{fb}}(y_k - \hat{y}_k). \end{aligned} \tag{22}$$

The model input is thus the recorded input plus the controller's response to the output error of the model. This form equals the model loop of (21) when the recorded input was produced by this controller from the same output samples and initial state. The difference of the controller states needs no estimate: setting it to zero at the window start places the model's controller in the machine's controller state, as the encoder places the model state on the recorded trajectory. Kessels' direct form instead needs the machine's controller state at every window start, read from the machine or reconstructed from the reference, the output and the controller [13, Remark 5.4].

The augmented model has no feedthrough, so each sample is evaluated in a fixed order: the output from the physical state, then the input, then the model and controller states advance. The controller feedthrough D_{fb} therefore closes no algebraic loop, and the loop adds no delay. In the coordinates of (15),

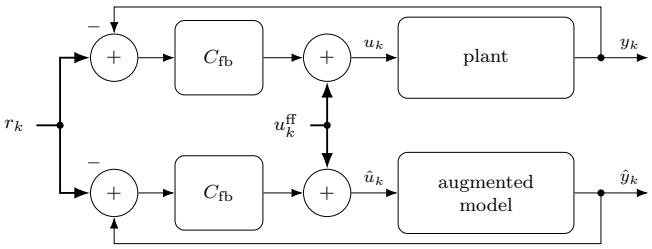


Fig. 4: Recorded plant loop (top) and augmented-model loop (bottom), driven by one shared reference r_k and feedforward u^{ff}_k, with the same feedback controller. The model output is evaluated before the input advances its state to the next sample.

where the output means cancel in the error, one step of a window started at τ is

$$\begin{aligned} \hat{y}^{\mathrm{n}}_{k|\tau} &= C^{\mathrm{n}} \tilde{x}^{\mathrm{n}}_{k|\tau}, \qquad \varepsilon_{k|\tau} = T_y^{-1}\big(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\big), \\ \hat{u}^{\mathrm{n}}_{k|\tau} &= u^{\mathrm{n}}_k + T_u\big(C_{\mathrm{fb}} x^c_k + D_{\mathrm{fb}} \varepsilon_{k|\tau}\big), \\ x^c_{k+1} &= A_{\mathrm{fb}} x^c_k + B_{\mathrm{fb}} \varepsilon_{k|\tau}, \qquad x^c_\tau = 0, \end{aligned} \tag{23}$$

after which $\hat{x}^{\mathrm{n}}_{k+1|\tau}$ follows from (13) with input $\hat{u}^{\mathrm{n}}_{k|\tau}$, starting from the encoder state (17). Here $\varepsilon_{k|\tau} = y_k - \hat{y}_{k|\tau}$ is the output error in metres. The output error of the model thus re-enters its own input through the controller, so a model error is partly corrected within the window, as on the machine. Model error made before τ is not carried into the window, whereas the free-run validation of Section III-D applies (23) over whole records and carries it.

D. Identification problem

We estimate θ_{base} jointly with the network and the encoder, as Hoekstra et al. do [8, Eq. (22)], whereas Kessels leaves the first-principles parameters fixed [13, Eq. (5.12)]. Every iterate must satisfy (12), so that $M(Y) \succ 0$ and the LFR stays well posed without a clamp or a penalty. We therefore train in coordinates $\xi \in \mathbb{R}^{10}$ that are zero at the start $\theta_{\mathrm{base},0}$. The nine positive combinations are logarithmic, $\theta_{\mathrm{base},j} = \theta_{\mathrm{base},0,j}\, e^{\xi_j}$, which keeps them positive. The signed m_Δ is bounded by (12) itself:

$$m_\Delta = \rho \, \frac{2}{L_b} \sqrt{m_\Sigma J_{\mathrm{eff}}} \, \tanh(\eta_0 + s\, \xi_\Delta), \tag{24}$$

where ξ_Δ is the component of ξ for m_Δ, $\rho \in (0,1)$ a margin, and η_0 and s reproduce $m_{\Delta,0}$ and its sensitivity at $\xi = 0$ (Appendix A-D). Every ξ thus gives $M(Y) \succ 0$ for all Y, which guarantees positive definite inertia but not positive raw masses.

Over the window starts $\mathcal{T}$ and the window length n_f, the trained parameters $\vartheta = (\xi, \theta_{\mathrm{aug}}, \theta_{\mathrm{enc}})$ minimise the normalised closed-loop output error, with the controller fixed. As in [8, Eq. (22)], every sample of a window is scored. We adapt that criterion in two respects: the model is simulated in the loop of Section III-C, and the encoder reads sample τ. Because the controller acts on the output error, a good closed-loop fit can

mask plant error, so Section VI also tests the model under a changed controller. The identification problem is

$$\begin{aligned}
&\min_{\vartheta}\ V(\vartheta) = \frac{1}{|\mathcal{T}|\, n_f\, n_y} \sum_{\tau\in\mathcal{T}} \sum_{k=\tau}^{\tau+n_f-1} \left\| y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau} \right\|_2^2 \quad \text{s.t.} \\
&\tilde{x}^{\mathrm{n}}_{k+1|\tau} = f^{\mathrm{n}}_{\mathrm{base}}\big(\theta_{\mathrm{base}}(\xi), \tilde{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big) + f^{\mathrm{n}}_{\mathrm{aug}}\big(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big), \\
&\bar{x}_{k+1|\tau} = g^{\mathrm{n}}_{\mathrm{aug}}\big(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big), \\
&\hat{x}^{\mathrm{n}}_{\tau|\tau} = \psi(\theta_{\mathrm{enc}}, \chi_\tau), \quad \hat{y}^{\mathrm{n}}_{k|\tau},\ \hat{u}^{\mathrm{n}}_{k|\tau} \text{ from (23)},
\end{aligned} \tag{25}$$

for $\tau \le k < \tau + n_f$, where $n_y = 3$ and $\theta_{\mathrm{base}}(\xi)$ is the coordinate map above. The controller matrices are fixed, but they are part of the computation: since $\hat{u}^{\mathrm{n}}_{k|\tau}$ depends on $\hat{y}^{\mathrm{n}}_{k|\tau}$, the gradient of V also passes through the controller recursion.

Because f_{aug} acts on the same rows as f_{base}, it can also learn changes that a different θ_{base} would produce, so the split between baseline and network is not unique [8, Sec. 5.2]. With a non-unique θ_{base}, the baseline can lose its physical meaning [11, Sec. 3]. Here, the estimated combinations are interpretable if they keep that meaning, which requires a unique estimate. Section IV extends the method so that this estimate is unique.

We minimise V with Adam on minibatches of windows, with the gradient obtained by automatic differentiation through the whole window, so no part of the window is truncated from the gradient. We then polish with L-BFGS on a fixed set of windows. Drenth et al. also follow Adam with L-BFGS, in its bounded form L-BFGS-B [16, Sec. 5]. Our polish needs no bounds, because (24) and the logarithmic coordinates give an admissible baseline for every ξ. During Adam, the parameters with the lowest validation error are kept, the closed-loop free-run error over whole validation records of Section V, each simulated from one encoder start with $x^c = 0$. The polish is kept only if it lowers this error and no monitored quantity regresses. The encoder history n, the window length n_f, the added-state dimension $n_{\bar{x}}$, the width and depth of ϕ_{aug} and the optimiser settings are given with their basis in Section V.

Training starts from the baseline, as Hoekstra et al. require of the initialisation [8, Eq. (29)]. Every trained block starts as

$$\begin{aligned}
&\xi = 0 \implies \theta_{\mathrm{base}} = \theta_{\mathrm{base},0}, \\
&W_L = 0,\ b_L = 0 \implies f^{\mathrm{n}}_{\mathrm{aug}} = 0,\ g^{\mathrm{n}}_{\mathrm{aug}} = 0, \\
&W^b \text{ from (20)}, \quad W^a \sim \mathcal{U}\big(-a, a\big), \quad \tilde{W}_L = 0,\ \tilde{b}_L = 0,
\end{aligned} \tag{26}$$

where $\theta_{\mathrm{base},0}$ are the starting combinations (Section V), $W_L \in \mathbb{R}^{(6+n_{\bar{x}})\times n_h}$ and $b_L \in \mathbb{R}^{6+n_{\bar{x}}}$ the weight and bias of the output layer of ϕ_{aug} with n_h its hidden width, $\tilde{W}_L$ and $\tilde{b}_L$ those of $\tilde{\psi}$, and $a = \sqrt{6/(n_{\bar{x}} + 6(n+1))}$ the Xavier bound, as in [8, Eq. (31)]. The hidden layers of ϕ_{aug} and $\tilde{\psi}$ keep the default random initialisation of the software. From the same physical initial state, the augmented model then reproduces the baseline.

We call $\mathcal{M}_{\mathrm{U}}$ the augmented model (13) identified by (25) from the start (26). Its input is the actuator force u_{act} and its output the actuator positions q_{act}.

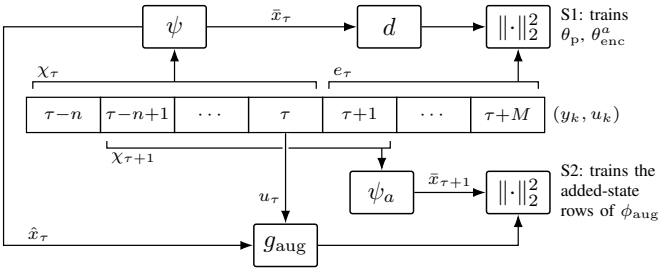


Fig. 5: Windows of PS on one record, layout adapted from [23, Fig. 2]. S1 (top): the encoder maps the window χ_τ to $\hat{x}_\tau$, and the decoder d maps its added part $\bar{x}_\tau$ to the residuals e_τ of the next M samples (27). S2 (bottom): g_{aug} maps $\hat{x}_\tau$ and the recorded u_τ to the added state encoded from the window shifted by one sample (29).

E. Added-state initialisation

The start (26) leaves the added states without a role. They are zero after the first step, and the output depends on them only through the zero output layer, so the gradient of V with respect to W^a and to the added-state rows of W_L is zero at the start. Minimising V can then lower the error through $f^{\mathrm{n}}_{\mathrm{aug}}$ alone, with static corrections or real lags, and leave the omitted dynamics unrepresented. Section VI tests whether it does. We therefore propose predictive state (PS) initialisation, which gives the added states a predictive role from data before the network reads them and then hands over to (25). It adapts the two-stage regression of predictive-state learning [23, Sec. 2] and its use as an initialisation before backpropagation through time [24, Sec. 4.2]. Unlike [23], the first stage predicts the residual of the model being trained rather than features of future observations, and the second stage regresses the nonlinear transition of the deployed model rather than a linear operator.

Fig. 5 shows which samples enter each stage. Stage S1 trains the encoder's added-state outputs to predict the residual of the model being trained.

During the first updates of (25), one S1 update with its own Adam optimiser follows every update of V. S1 draws windows from the training records except a calibration subset of records. Its target is the current model's closed-loop residual over the $M = n + 1$ samples from the window start, and a temporary decoder reads only $\bar{x}_\tau$:

$$\begin{aligned}
&e_\tau = \frac{1}{\sigma_e} \operatorname{col}\big(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\big)_{k=\tau}^{\tau+M-1}, \\
&\min_{\theta_{\mathrm{p}},\, \theta^a_{\mathrm{enc}}} \ \frac{1}{|\mathcal{B}|\, M\, n_y} \sum_{\tau\in\mathcal{B}} \big\| d\big(\theta_{\mathrm{p}}, \psi_a(\theta_{\mathrm{enc}}, \chi_\tau)\big) - e_\tau \big\|_2^2 ,
\end{aligned} \tag{27}$$

where $\hat{y}^{\mathrm{n}}_{k|\tau}$ is the output of the loop of Section III-C at the current ϑ, held fixed in the gradient, $\mathcal{B}$ a minibatch of window starts, σ_e the RMS of the residuals over $\mathcal{B}$, and d a decoder with parameters θ_{p} that maps the $n_{\bar{x}}$ added states to the Mn_y residual samples, with one tanh layer of the width of ϕ_{aug} and a zero output layer. Only θ_{p} and θ^a_{enc} receive a gradient, so the rest of ϑ is left to V. Sample τ is both the last sample of

the encoder window and the first of the target, so S1 predicts exactly the residual sequence that V scores.

Whether $\bar{x}_\tau$ predicts the residual is measured on a fixed set $\mathcal{C}$ of windows from the calibration records, by the fraction the decoder leaves unexplained,

$$\nu = \frac{\sum_{\tau\in\mathcal{C}} \left\| d\big(\theta_{\mathrm{p}}, \psi_a(\theta_{\mathrm{enc}}, \chi_\tau)\big) - e_\tau \right\|_2^2}{\sum_{\tau\in\mathcal{C}} \|e_\tau\|_2^2}. \tag{28}$$

S1 ends when ν stops falling, or after a maximum number of updates. If ν then exceeds a threshold, or already does at a fixed early update, the residual is not predictable from $\bar{x}_\tau$: S2 is skipped and training continues with the S1 changes kept. Otherwise, stage S2 fits the added-state transition so that it propagates these states. Once, at the end of S1, we encode window pairs that start at samples k_i and $k_i + 1$ anywhere in the fit records, freeze them as the set $\mathcal{P}$, and fit $g^{\mathrm{n}}_{\mathrm{aug}}$ by one-step regression with a fresh optimiser, keeping the best parameters on pairs from the calibration records:

$$\min_{\theta_{\mathrm{aug}}} \frac{\sum_{i\in\mathcal{P}} \left\| g^{\mathrm{n}}_{\mathrm{aug}}\big(\theta_{\mathrm{aug}}, \psi(\theta_{\mathrm{enc}}, \chi_{k_i}), u^{\mathrm{n}}_{k_i}\big) - \psi_a(\theta_{\mathrm{enc}}, \chi_{k_i+1}) \right\|_2^2}{\sum_{j=1}^{n_{\bar{x}}} \sum_{i\in\mathcal{P}} \big(\psi_{a,j}(\theta_{\mathrm{enc}}, \chi_{k_i+1}) - \bar{\psi}_{a,j}\big)^2}, \tag{29}$$

minimised on minibatches of $\mathcal{P}$, where θ_{enc} is held at its value at the end of S1 and $\bar{\psi}_{a,j}$ is the mean of the jth target. The input $u^{\mathrm{n}}_{k_i}$ is the recorded one: the force that moved the machine from k_i to $k_i + 1$, from whose response the target is encoded. At a window start of the loop (22), $x^c = 0$, so the model input there is

$$\hat{u}_{\mathrm{act},k_i} = u_{\mathrm{act},k_i} + D_{\mathrm{fb}}\big(y_{k_i} - \hat{y}_{k_i}\big),$$

which equals the recorded input when the encoder reproduces y_{k_i}. The loss reaches ϕ_{aug} only through its added-state rows, so S2 leaves the physical rows of W_L unchanged, while the shared hidden layers move.

In stage S3 the decoder is discarded and the minimisation of (25) continues from the modified parameters, followed by the polish and selection of Section III-D. PS adds nothing to the deployed model, the criterion or the selection rule. The added states remain latent, since PS assigns them no physical coordinates. An invertible transformation of $\bar{x}$, absorbed by ϕ_{aug}, leaves the input-output behaviour unchanged. We call $\mathcal{M}_{\mathrm{PS}}$ the model (13) identified by (25) with this opening, from the same start as $\mathcal{M}_{\mathrm{U}}$.

IV. Interpretability by Orthogonal Construction

The split between baseline and network in the augmented model of Section III-D is not unique, and our goal is an estimate of θ_{base} that is unique under stated conditions. To this end, we construct an orthogonal correction of the physical rows (Section IV-A) and state the identification problem of the corrected model (Section IV-B). We then give the condition under which its estimate equals the true combinations, with the scope of this result (Section IV-C). This delivers the third contribution of Section I: state-level orthogonality for the identifiable physical-parameter tangent space and its true-parameter recovery condition.

A. Orthogonal correction of the physical rows

The non-unique split lets the network negate the baseline, Kessels' term for learned terms that partially cancel the first-principles terms [13, Sec. 5.3.1.3]. Let $\xi' \in \mathbb{R}^{10}$ be other values of the training coordinates ξ of (24). The network term $f^{\mathrm{n}}_{\mathrm{aug}} - (\partial f^{\mathrm{n}}_{\mathrm{base}}/\partial\xi)(\xi' - \xi)$ with ξ' then gives, to first order in $\xi' - \xi$, the same one-step map as $f^{\mathrm{n}}_{\mathrm{aug}}$ with ξ. On the gantry, each column of $\partial f^{\mathrm{n}}_{\mathrm{base}}/\partial\xi$ is the RK4 propagation of the acceleration change $-M(Y)^{-1}\big(\frac{\partial M}{\partial \xi_j}\ddot{q} + \frac{\partial C}{\partial \xi_j}\dot{q} + \frac{\partial K}{\partial \xi_j} q\big)$ of (2). A negating term therefore acts as an added mass, damping or stiffness force whose effect on the state depends on Y.

The research plan proposed the orthogonality penalty of Györök et al., whose weight trades orthogonality against fit [25, Eq. (13), Remark 1]. We instead adopt their later construction, which makes the network term orthogonal to the baseline without a weight [11, Sec. 3.2]. It needs a baseline linear in its parameters [11, Eq. (2)]. Our transition is rational in θ_{base} through $M(Y)^{-1}$, so it has no linear-in-parameters form.

We therefore linearise the normalised transition in its parameters, as Györök et al. do for a nonlinear baseline [25, Sec. 4, Eq. (15)]. We differentiate with respect to ξ rather than the fourteen raw parameters, whose four lost directions (Section II-C) would give the Jacobian four null directions. The map (24) has a nonsingular Jacobian for $m_{\Delta,0} \neq 0$, so the Jacobian in ξ spans the sensitivity directions of the ten combinations θ_{base}. Unlike their extended regressor [25, Eq. (18)], we omit the offset column $f^{\mathrm{n}}_{\mathrm{base}}$ itself, for two reasons. The estimate of ξ moves only within the range of the Jacobian (Section IV-C), so the offset would constrain directions that do not move it. A parameter derivative also does not change when the origin of the normalised state moves, whereas the offset does, so the protected directions would depend on the centring μ_x of (15). The sensitivity is

$$\Phi(\xi_{\mathrm{lin}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}) = \left.\frac{\partial f^{\mathrm{n}}_{\mathrm{base}}\big(\theta_{\mathrm{base}}(\xi), \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}\big)}{\partial\xi}\right|_{\xi=\xi_{\mathrm{lin}}}, \tag{30}$$

where $\xi_{\mathrm{lin}} \in \mathbb{R}^{10}$ is the linearisation point and $\Phi : \mathbb{R}^{10} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^{6\times 10}$.

We evaluate Φ on fixed reference tuples from the training records, because only the positions of the baseline state are measured. Each tuple holds the positions $q_i = P^{-\top} q_{\mathrm{act},i}$ of a training sample, velocities from a fourth-order central difference of q within the record, $\bar{x}_i = 0$ and the recorded input. The tuples cover every sample at which this difference exists. Each tuple is one step at the recorded input, so the controller does not enter. Györök et al. simulate the baseline when the states are not measured [25, Sec. 3]. In open loop, such a simulation drifts on the free axes (Section III-C).

Adapting the coefficient of Györök et al. to these tuples [11, Eqs. (8), (11), Lemma 3], we fit the network's physical rows by the stacked sensitivity in the least-squares sense. As they assume of their regressor [11, Assumption 1], the stack has

full column rank:

$$\begin{aligned}\boldsymbol{\Phi}(\xi_{\mathrm{lin}}) &= \mathrm{col}\big(\Phi(\xi_{\mathrm{lin}}, \tilde{x}_i^{\mathrm{n}}, u_i^{\mathrm{n}})\big)_{i\in\mathcal{I}},\\ \boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}}(\theta_{\mathrm{aug}}) &= \mathrm{col}\big(f_{\mathrm{aug}}^{\mathrm{n}}(\theta_{\mathrm{aug}}, \mathrm{col}(\tilde{x}_i^{\mathrm{n}}, 0), u_i^{\mathrm{n}})\big)_{i\in\mathcal{I}},\\ \theta_{\mathrm{aux}}(\theta_{\mathrm{aug}}) &= \boldsymbol{\Phi}(\xi_{\mathrm{lin}})^{+}\, \boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}}(\theta_{\mathrm{aug}}),\\ 0 &= \boldsymbol{\Phi}(\xi_{\mathrm{lin}})^{\top}\big(\boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}}(\theta_{\mathrm{aug}}) - \boldsymbol{\Phi}(\xi_{\mathrm{lin}})\,\theta_{\mathrm{aux}}(\theta_{\mathrm{aug}})\big),\end{aligned} \tag{31}$$

where $\mathcal{I}$ is the set of tuples, $N = |\mathcal{I}|$, $\tilde{x}_i^{\mathrm{n}} \in \mathbb{R}^6$ and $u_i^{\mathrm{n}} \in \mathbb{R}^3$ are the state and input of tuple i in the coordinates of (15), $\boldsymbol{\Phi}(\xi_{\mathrm{lin}}) \in \mathbb{R}^{6N\times 10}$, $\boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}}(\theta_{\mathrm{aug}}) \in \mathbb{R}^{6N}$, $^{+}$ is the Moore-Penrose pseudoinverse and $\theta_{\mathrm{aux}} \in \mathbb{R}^{10}$.

As in [11, Eqs. (9), (13)], the coefficient is a function of the network parameters, and every prediction subtracts the sensitivity times the coefficient. We subtract it from the physical rows at every step of the augmented model, at the state and input of that step. The added-state rows are not corrected, because the baseline has no added-state update to be orthogonal to. The correction is

$$f_{\mathrm{obc}}^{\mathrm{n}}(\theta_{\mathrm{aug}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}) = \Phi(\xi_{\mathrm{lin}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}})\,\theta_{\mathrm{aux}}(\theta_{\mathrm{aug}}), \tag{32}$$

where $f_{\mathrm{obc}}^{\mathrm{n}} : \mathbb{R}^{n_{\mathrm{aug}}} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^6$. Differentiating $\theta_{\mathrm{aux}}(\theta_{\mathrm{aug}})$ gives the network the gradient of the corrected term, which a detached coefficient would replace by that of the uncorrected one.

We set ξ_{lin} to the current ξ at each epoch boundary, rebuild $\boldsymbol{\Phi}(\xi_{\mathrm{lin}})$ there and hold it for the epoch. No gradient therefore reaches ξ through Φ. This adapts the per-epoch refresh that Györök et al. allow in their linear case [25, Remark 2] to the linearisation point. For a nonlinear baseline, they update this point at every evaluation of the objective or fix it at the nominal parameters [25, Sec. 4]. The stacked sensitivity changes little within an epoch, so a rebuild at every evaluation would mainly add cost.

B. Identification of the corrected model

The identification problem of Section III-D must now include the correction in the physical-row update. The trained parameters $\vartheta = (\xi, \theta_{\mathrm{aug}}, \theta_{\mathrm{enc}})$ and the cost V are unchanged:

$$\begin{aligned}&\min_{\vartheta}\ V(\vartheta) \quad \text{s.t.}\\ &\tilde{x}_{k+1|\tau}^{\mathrm{n}} = f_{\mathrm{base}}^{\mathrm{n}}\big(\theta_{\mathrm{base}}(\xi), \tilde{x}_{k|\tau}^{\mathrm{n}}, \hat{u}_{k|\tau}^{\mathrm{n}}\big) + f_{\mathrm{aug}}^{\mathrm{n}}\big(\theta_{\mathrm{aug}}, \hat{x}_{k|\tau}^{\mathrm{n}}, \hat{u}_{k|\tau}^{\mathrm{n}}\big)\\ &\qquad\qquad - f_{\mathrm{obc}}^{\mathrm{n}}\big(\theta_{\mathrm{aug}}, \tilde{x}_{k|\tau}^{\mathrm{n}}, \hat{u}_{k|\tau}^{\mathrm{n}}\big),\\ &\text{the other constraints of (25),}\end{aligned} \tag{33}$$

where ξ_{lin} in (32) is held fixed during each update and refreshed per epoch (Section IV-A), and θ_{aux} is solved by (31) from the current θ_{aug} at every evaluation of V.

The optimiser, the stages of Section III-E and the selection rule of Section III-D are reused. Each S1 update fits the residual of the corrected model. It changes only θ_{p} and $\theta_{\mathrm{enc}}^{a}$. S2 fits $g_{\mathrm{aug}}^{\mathrm{n}}$, which the correction does not touch. It changes θ_{aug}, so θ_{aux} changes at the next evaluation of V. The Adam updates of V, during S1 and in S3, change every entry of ϑ. Validation uses the basis of the last epoch. After the best Adam checkpoint is restored, the basis is rebuilt at its ξ. The polish then changes every entry of ϑ with this basis, under the acceptance rule of Section III-D.

We call $\mathcal{M}_{\mathrm{OBC}}$ the model $\mathcal{M}_{\mathrm{PS}}$ with the correction (32), identified by (33). Its input is the actuator force u_{act} and its output the actuator positions q_{act}. It starts as $\mathcal{M}_{\mathrm{PS}}$ does, with the first basis built at $\xi = 0$. Section VI compares $\mathcal{M}_{\mathrm{OBC}}$ with $\mathcal{M}_{\mathrm{PS}}$ on the ten combinations and on accuracy.

C. Scope and condition for true parameter recovery

Orthogonality on the tuples does not yet say whether the estimate is unique, nor when it equals the true combinations. Recovery depends on what the baseline misses in the data. Let $\xi^{\star} \in \mathbb{R}^{10}$ be the coordinates of the true combinations. The successor of tuple i is tuple $i+1$ of the same record, which every tuple except the last of each record has. On the set $\mathcal{I}' \subset \mathcal{I}$ of tuples with a successor, the true one-step discrepancy is

$$\varepsilon = \tilde{\boldsymbol{x}}_{+}^{\mathrm{n}} - \boldsymbol{f}_{\mathrm{base}}^{\mathrm{n}}(\xi^{\star}), \qquad \tilde{\boldsymbol{x}}_{+}^{\mathrm{n}} = \mathrm{col}\big(\tilde{x}_{i+1}^{\mathrm{n}}\big)_{i\in\mathcal{I}'}, \tag{34}$$

where $\boldsymbol{f}_{\mathrm{base}}^{\mathrm{n}}(\xi) = \mathrm{col}\big(f_{\mathrm{base}}^{\mathrm{n}}(\theta_{\mathrm{base}}(\xi), \tilde{x}_i^{\mathrm{n}}, u_i^{\mathrm{n}})\big)_{i\in\mathcal{I}'}$ and $\varepsilon, \tilde{\boldsymbol{x}}_{+}^{\mathrm{n}} \in \mathbb{R}^{6|\mathcal{I}'|}$. Since the tuple velocities are differenced positions, ε also holds the error of this reconstruction.

We adapt the recovery theorem of Györök et al., with its orthogonality condition and parameter error [11, Cond. 4, Assumption 6, Thm. 7, Eq. (21)]. Their theorem assumes that the model reproduces the data exactly. The least-squares form below needs no exact fit. It concerns the variant of (31) in which every stack runs over $\mathcal{I}'$.

Proposition 1: Suppose (i) $\boldsymbol{\Phi}(\xi_{\mathrm{lin}})$ has full column rank, and (ii) $\boldsymbol{f}_{\mathrm{base}}^{\mathrm{n}}(\xi) = \boldsymbol{f}_{\mathrm{base}}^{\mathrm{n}}(\xi^{\star}) + \boldsymbol{\Phi}(\xi_{\mathrm{lin}})(\xi - \xi^{\star})$ for every ξ, which the gantry meets to first order in $\xi - \xi^{\star}$ and $\xi_{\mathrm{lin}} - \xi^{\star}$. Then, for every θ_{aug}, the one-step error $\big\|\tilde{\boldsymbol{x}}_{+}^{\mathrm{n}} - \boldsymbol{f}_{\mathrm{base}}^{\mathrm{n}}(\xi) - \boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}}(\theta_{\mathrm{aug}}) + \boldsymbol{\Phi}(\xi_{\mathrm{lin}})\theta_{\mathrm{aux}}(\theta_{\mathrm{aug}})\big\|_2^2$ has the same unique minimiser over ξ, $\xi = \xi^{\star} + \boldsymbol{\Phi}(\xi_{\mathrm{lin}})^{+}\varepsilon$, which does not depend on the network. It equals $\xi^{\star}$ if and only if $\boldsymbol{\Phi}(\xi_{\mathrm{lin}})^{\top}\varepsilon = 0$.

Proof: Write $\boldsymbol{\Phi}$ for $\boldsymbol{\Phi}(\xi_{\mathrm{lin}})$ and $\Pi = \boldsymbol{\Phi}\boldsymbol{\Phi}^{+}$, the orthogonal projector onto $\mathrm{range}\,\boldsymbol{\Phi}$. By (ii), (34) and (31), the stacked residual is $\varepsilon - \boldsymbol{\Phi}(\xi - \xi^{\star}) - (I - \Pi)\boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}}$. Its projections by Π and $I - \Pi$ are $\Pi\varepsilon - \boldsymbol{\Phi}(\xi - \xi^{\star})$ and $(I - \Pi)(\varepsilon - \boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}})$. They are orthogonal, so the one-step error is $\|\Pi\varepsilon - \boldsymbol{\Phi}(\xi - \xi^{\star})\|_2^2 + \big\|(I - \Pi)(\varepsilon - \boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}})\big\|_2^2$. The second term does not depend on ξ. The first vanishes if and only if $\boldsymbol{\Phi}(\xi - \xi^{\star}) = \Pi\varepsilon$, which by (i) holds only for $\xi - \xi^{\star} = \boldsymbol{\Phi}^{+}\varepsilon$. By (i), $\boldsymbol{\Phi}^{+} = (\boldsymbol{\Phi}^{\top}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\top}$, so $\boldsymbol{\Phi}^{+}\varepsilon = 0$ if and only if $\boldsymbol{\Phi}^{\top}\varepsilon = 0$. ∎

Without the correction, the same steps give $\xi - \xi^{\star} = \boldsymbol{\Phi}^{+}(\varepsilon - \boldsymbol{f}_{\mathrm{aug}}^{\mathrm{n}})$, which moves with the network along the directions of Section IV-A.

The condition $\boldsymbol{\Phi}^{\top}\varepsilon = 0$ is a condition on the excitation. Györök et al. note that an input can be designed to meet their condition when the structure of the omitted terms is known [11, Remark 5]. Without it, Proposition 1 gives a unique estimate, to first order, but not the true combinations. Because ε holds the reconstruction error, the condition can fail even where the baseline misses nothing. Proposition 1 also concerns the one-step error on the tuples, whereas (33) minimises a closed-loop multistep output error. It is therefore a surrogate, not the bias of the trained model. No orthogonality of trajectories

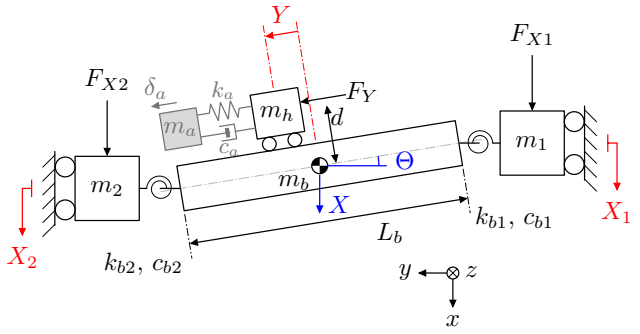


Fig. 6: Simulated gantry with a payload-mounted absorber. The light-gray elements $(m_a, k_a, c_a, \delta_a)$ are absent from the baseline.

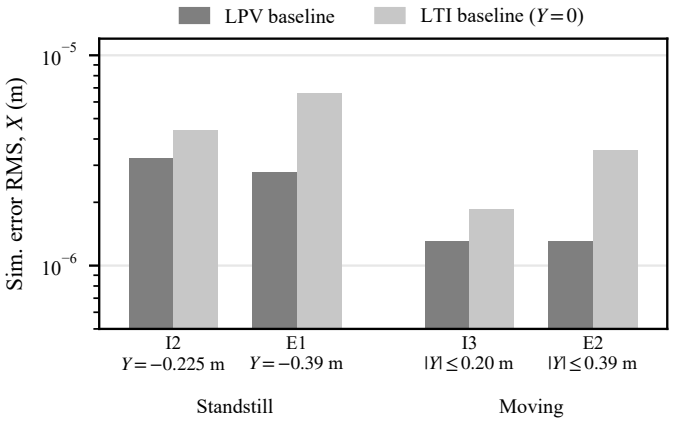


Fig. 7: Simulation error RMS on the X axes of the LPV baseline and the LTI baseline at $Y = 0$, both at the true parameters.

or outputs follows from it. The tuples lie on $\bar{x} = 0$, so θ_{aux} sees the network only there. With the added states active, the part of $f^{\text{n}}_{\text{aug}}$ that acts through $\bar{x}$ is not protected. The uncorrected $g^{\text{n}}_{\text{aug}}$ reaches the physical rows one step later through $f^{\text{n}}_{\text{aug}}$. Without the offset column, a network term that cancels the nominal transition $f^{\text{n}}_{\text{base}}$ is charged only through its component in $\operatorname{range}\mathbf{\Phi}$. It does not move the estimate, because the minimiser of Proposition 1 does not depend on $\boldsymbol{f}^{\text{n}}_{\text{aug}}$. The inner product is Euclidean in the coordinates of (15), so the scaling T_x weights the six rows. Finally, the consistency and covariance results of Györök et al. do not transfer, because they concern an input-output baseline linear in its parameters [11, Secs. 4.3, 4.4].

V. Experiment Design

A. System and data

- System: LPV baseline plus a payload absorber (50 % of the head mass, 150 Hz, $\zeta = 0.03$) plus tanh Coulomb rail friction, in closed loop with controller K1 (Fig. 6)
- Choice: absorber and friction only in the system, as known missing dynamics
- Choice: output noise at the measured machine level; noise floor = noisy record minus its noise-free twin
- Excitation: multisine of 106 to 297 Hz
- Types of motion: standstill at a fixed Y, slow Y sweeps, X-Y Lissajous paths, S-curve point-to-point moves, sine-shaped
- Training (18 records): fits the model; Y within ± 0.30 m, accelerations up to 75 % of the machine maximum
- Validation (6 records): only selects the checkpoint and the black-box size; never reported
- Test (12 records): every reported number
- Interpolation (I1 to I6): inside the training range, with new multisine phases, new Y positions and new move sequences (sine and S-curve routes, X-only and Y-only moves, long strokes)
- Extrapolation (E1 to E6): each differs from its I record in one property outside the training range: Y position (E1, E2, up to 0.39 m), acceleration (E3, 100 %), velocity (E4, 1.95 m/s), setpoint using movement from machine (E6), a changed controller (E5, not done currently)

B. Compared models

- Baseline: LPV at the true and at the detuned parameters; LTI frozen at $Y = 0$
- Augmented: static ($n_a = 0$), plain training, PS initialisation ($n_a = 2$ and 8), unconstrained against OBC
- Choice: start 10 % detuned for the combinations, the same direction in every run
- Choice: black box = SUBNET on the closed loop $(r, f) \to e$, size selected from 11 configurations on validation

C. Training and evaluation

- Closed-loop training, window 0.1 s, encoder history 29 samples, network 2×16 tanh
- Choice: one seed per run (compute limit)
- Metric: simulation error RMS per axis in m, over the whole test record
- Validation only selects checkpoints; every reported number is on the test set

VI. Results

A. Baseline form

- Criterion: LPV error flat over Y, LTI error growing away from $Y = 0$ (Fig. 7)
- On X the LTI error is 1.5 to 3.7 times the LPV error, growing with $|Y|$; on Y both are equal
- Y scheduling is needed for the X axes

B. Augmentation structure

- Criterion: error below the baseline and below the model without added states (Table I)
- PS initialisation lowers the Y error about 9 times; plain training and $n_a = 0$ stay near the baseline
- The error in the multisine band disappears with PS initialisation, not with plain training (Fig. 8)

TABLE I: Simulation error RMS per axis, mean over test records I1 to I6.

Model	X_1 (m)	X_2 (m)	Y (m)
Baseline, true θ	$1.64 \cdot 10^{-6}$	$1.97 \cdot 10^{-6}$	$1.31 \cdot 10^{-5}$
Baseline, detuned θ	$2.99 \cdot 10^{-6}$	$2.28 \cdot 10^{-6}$	$1.56 \cdot 10^{-5}$
Static parallel, n_a=0	$1.33 \cdot 10^{-6}$	$1.42 \cdot 10^{-6}$	$1.24 \cdot 10^{-5}$
Plain training, n_a=2	$1.18 \cdot 10^{-6}$	$1.26 \cdot 10^{-6}$	$1.06 \cdot 10^{-5}$
PS init., n_a=2	$1.09 \cdot 10^{-6}$	$1.21 \cdot 10^{-6}$	$1.39 \cdot 10^{-6}$
PS init., baseline part only	$1.94 \cdot 10^{-6}$	$1.77 \cdot 10^{-6}$	$1.35 \cdot 10^{-5}$
PS init., n_a=8	$1.04 \cdot 10^{-6}$	$1.19 \cdot 10^{-6}$	$1.44 \cdot 10^{-6}$
PS init., detuned θ	$1.08 \cdot 10^{-6}$	$1.20 \cdot 10^{-6}$	$1.44 \cdot 10^{-6}$
Augmented (OBC), detuned θ	$1.14 \cdot 10^{-6}$	$1.29 \cdot 10^{-6}$	$1.49 \cdot 10^{-6}$
Black-box model	$1.85 \cdot 10^{-5}$	$2.11 \cdot 10^{-5}$	$2.33 \cdot 10^{-5}$
Noise floor	$9.00 \cdot 10^{-9}$	$9.63 \cdot 10^{-9}$	$3.19 \cdot 10^{-9}$

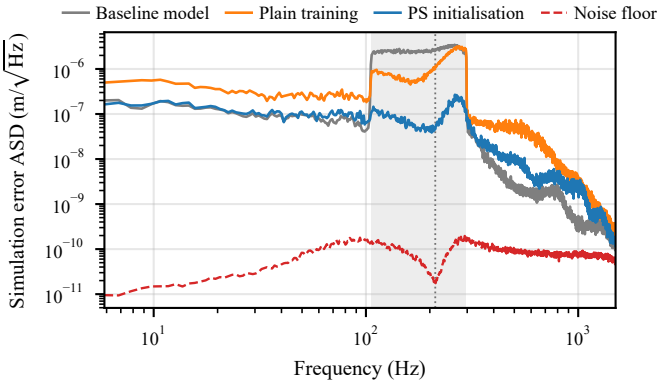


Fig. 8: Amplitude spectral density of the Y simulation error on test records I1 and I2. Shaded: multisine band. Dotted: absorber pole.

- The learned pole is at 212 Hz, the absorber pole of the system at 212 Hz; $n_a = 8$ adds nothing
- Without added states ($n_a = 0$) the Y error stays at the baseline level
- Time trace of the augmented model and the system (Fig. 9)

C. Interpretability

- Criterion: OBC drifts less than the unconstrained arm at equal test RMS (Table II)
- Not met: the parameters drift in both arms, OBC not less
- OBC keeps the learned term out of the parameter directions: overlap 0.12 to 0.13 against 0.91 to 0.99
- Cause: 59 % of the missing dynamics lies in the parameter directions

D. Generalisation

- Criterion: at least as good as the black-box model on I, better on E (Fig. 10)
- Met: the augmented model is below the detuned baseline on all records, and 10 to 30 times below the black-box model on I and E
- During motion about 10^{-6} m remains, 100 times above the noise floor

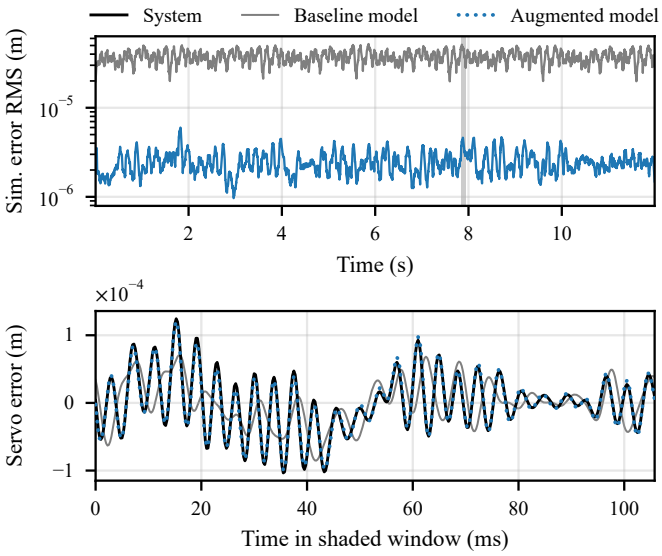


Fig. 9: Test record I1, Y axis. Top: moving RMS of the simulation error. Bottom: servo error in the shaded window.

TABLE II: Relative error of the ten combinations, unconstrained and OBC, from the detuned and the correct start.

	Detuned start			Correct start	
Rel. error (%)	Start	Unconstr.	OBC	Unconstr.	OBC
$k_{b,\Sigma}$	−10.0	−2.3	−6.2	−9.2	−2.7
c_{g1}	+10.0	+35.1	+45.0	+16.4	+28.3
c_{g2}	+10.0	+44.7	+44.1	+29.4	+35.3
c_y	+10.0	+41.5	+12.6	+30.0	−6.0
$c_{b,\Sigma}$	−10.0	+15.2	+19.9	+28.8	+34.7
m_h	−10.0	−3.3	−3.6	−2.2	−2.2
m_Σ	+10.0	+2.4	+2.5	+3.1	+2.0
m_Δ	+10.0	+33.0	+44.5	+14.4	+34.5
J_{eff}	−10.0	−0.8	−1.9	+3.8	+2.6
d	−10.0	−3.2	+10.4	−9.0	+14.9
Norm	31.6	79.4	81.7	57.1	68.7
Test RMS ($\times 10^{-6}$ m)		1.24	1.31	1.23	1.30
Learned-term overlap		0.99	0.12	0.91	0.13
Missing-dynamics overlap			0.59		
Learned pole (Hz)		212	213	212	214
Learned damping ζ		0.049	0.052	0.051	0.052

VII. Conclusions and Discussion

A. Conclusions

- Aspect 1: LPV scheduling lowers the error on X, increasingly away from $Y = 0$
- Aspect 2: added states with PS initialisation learn the missing mode; plain training does not
- Aspect 3: OBC keeps the learned term out of the parameter directions; the parameters still drift
- Aspect 4: better than the black-box model on interpolation and extrapolation
- Black box currently doesnt perform well on test sets.
- Recommendation (?)

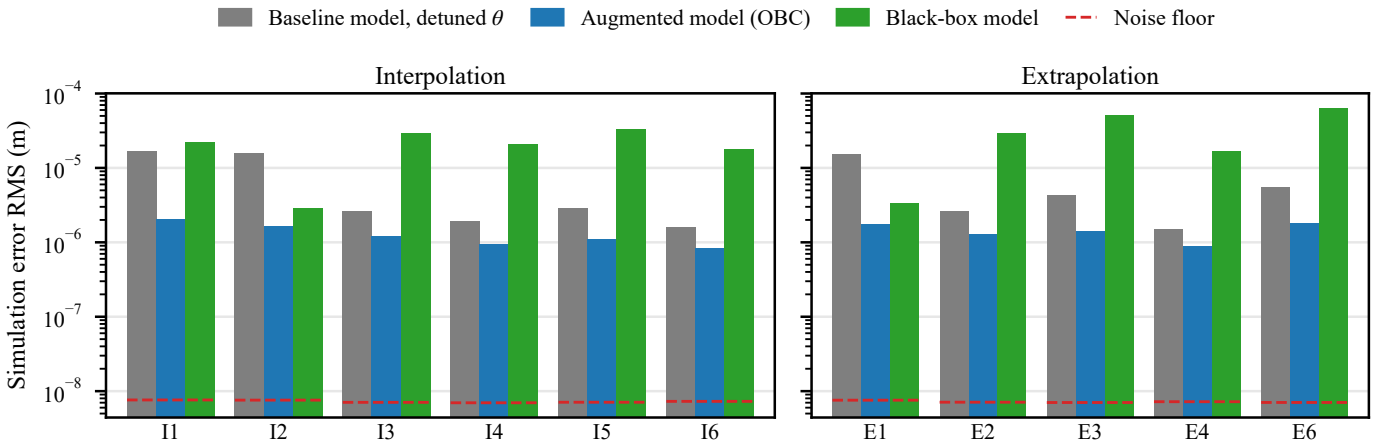


Fig. 10: Simulation error RMS per test record, mean over X_1, X_2, Y.

B. Discussion

- Why the parameters drift despite OBC (missing-dynamics overlap)
- What the black-box comparison does and does not show
- The error during motion is not captured, likely friction??
- Single seed, simulation only, no controller transfer
- OBC holds only at $\bar{x} = 0$; current definition doesn't take into account the additional states entering the output the next update step.
- Need to fix: current enencoder is initialized with true parameters
- Future work: friction, excitation for parameter recovery, real data

References

[1] International Roadmap for Devices and Systems (IRDS), "International roadmap for devices and systems: 2022 edition executive summary," IEEE, Tech. Rep., 2022.

[2] T. Workman, L. Mirkarimi, J. Theil, G. Fountain, K. M. Bang, B. Lee, C. Uzoh, D. Suwito, G. Gao, and P. Mrozek, "Die to wafer hybrid bonding and fine pitch considerations," in *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*. IEEE, 2021, pp. 2071–2077.

[3] I. García-Herreros, X. Kestelyn, J. Gomand, R. Coleman, and P.-J. Barre, "Model-based decoupling control method for dual-drive gantry stages: A case study with experimental validations," *Control Engineering Practice*, vol. 21, no. 3, pp. 298–307, 2013.

[4] C. S. Teo, K. K. Tan, S. Y. Lim, S. Huang, and E. B. Tay, "Dynamic modeling and adaptive control of a h-type gantry stage," *Mechatronics*, vol. 17, no. 7, pp. 361–367, 2007.

[5] P. Lambrechts, M. Boerlage, and M. Steinbuch, "Trajectory planning and feedforward design for electromechanical motion systems," *Control Engineering Practice*, vol. 13, no. 2, pp. 145–157, 2005.

[6] J. Hwang and S. D. Noh, "Digital twin-based optimization of operational parameters for cluster tools in semiconductor manufacturing," *IEEE Access*, vol. 12, pp. 122 078–122 100, 2024.

[7] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating scientific knowledge with machine learning for engineering and environmental systems," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–37, Nov. 2022.

[8] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Learning-based augmentation of first-principle models: A linear fractional representation-based approach," 2026, submitted to Automatica.

[9] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, Jun. 2021.

[10] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations," *arXiv preprint arXiv:1711.10561*, 2017.

[11] B. M. Györök, M. Schoukens, T. Péni, and R. Tóth, "Orthogonal-by-construction augmentation of physics-based input-output models," *IFAC Journal of Systems and Control*, vol. 35, p. 100376, 2026.

[12] J. P. Noël and J. Schoukens, "Grey-box state-space identification of nonlinear mechanical vibrations," *International Journal of Control*, vol. 91, no. 5, pp. 1118–1139, 2018.

[13] B. M. Kessels, "Ai-based model updating for nonlinear dynamical systems," PhD thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2025.

[14] A. Retzler, R. Tóth, M. Schoukens, G. I. Beintema, J. Weigand, J.-P. Noël, Z. Kollár, and J. Swevers, "Learning-based augmentation of physics-based models: an industrial robot use case," *Data-Centric Engineering*, vol. 5, p. e12, 2024.

[15] R. Tóth, *Modeling and Identification of Linear Parameter-Varying Systems*, ser. Lecture Notes in Control and Information Sciences. Berlin, Heidelberg: Springer, 2010, vol. 403.

[16] R. Drenth, J. H. Hoekstra, M. Schoukens, and R. Tóth, "Efficient learning of affine and rational dependency LPV models with linear fractional representation," 2026.

[17] A. Ovchinnikov, A. Pillay, G. Pogudin, and T. Scanlon, "Computing all identifiable functions of parameters for ODE models," *Systems & Control Letters*, vol. 157, p. 105030, 2021.

[18] M. Gautier and W. Khalil, "Direct calculation of minimum set of inertial parameters of serial robots," *IEEE Transactions on Robotics and Automation*, vol. 6, no. 3, pp. 368–373, 1990.

[19] R. Drenth, "Efficient gradient-based learning of LPV models with linear fractional representation," Master's thesis, Eindhoven University of Technology, 2025.

[20] G. I. Beintema, M. Schoukens, and R. Tóth, "Deep subspace encoders for nonlinear system identification," *Automatica*, vol. 156, p. 111210, 2023.

[21] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Encoder initialisation methods in the model augmentation setting," 2026.

[22] U. Forssell and L. Ljung, "Closed-loop identification revisited," *Automatica*, vol. 35, no. 7, pp. 1215–1241, 1999.

[23] A. Hefny, C. Downey, and G. J. Gordon, "Supervised learning for dynamical system learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[24] C. Downey, A. Hefny, B. Li, B. Boots, and G. J. Gordon, "Predictive state recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[25] "Source pending verification," citation metadata not yet verified.

APPENDIX A
SUPPORTING DERIVATIONS

A. Explicit LPV-LFR realization

In (5), M_0 is (3) evaluated at $Y = 0$, and

$$M_1 = \begin{bmatrix} 0 & -m_h & 0 \\ -m_h & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad M_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & m_h & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

With a_1 and a_2 from (7) treated as independent signals, solving $M_0\ddot{q} + M_1 a_1 + M_2 a_2 = u - C\dot{q} - Kq$ for $\ddot{q}$ with $R = M_0^{-1}$ gives $\ddot{q} = R(u - Kq - C\dot{q} - M_1 a_1 - M_2 a_2)$. With $\tilde{x} = \mathrm{col}(q, \dot{q})$, $z = \mathrm{col}(\ddot{q}, a_1)$, and $w = \mathrm{col}(a_1, a_2)$, the LTI part G of (10), partitioned as in [16, Eq. (6)], is

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = \underbrace{\begin{bmatrix} A_x & B_w & B_u \\ C_z & D_{zw} & D_{zu} \\ C_y & D_{yw} & D_{yu} \end{bmatrix}}_{G} \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}. \tag{35}$$

with blocks

$$\begin{aligned} A_x &= \begin{bmatrix} 0 & I_3 \\ -RK & -RC \end{bmatrix}, & B_w &= \begin{bmatrix} 0 & 0 \\ -RM_1 & -RM_2 \end{bmatrix}, & B_u &= \begin{bmatrix} 0 \\ R \end{bmatrix}, \\ C_z &= \begin{bmatrix} -RK & -RC \\ 0 & 0 \end{bmatrix}, & D_{zw} &= \begin{bmatrix} -RM_1 & -RM_2 \\ I_3 & 0 \end{bmatrix}, & D_{zu} &= \begin{bmatrix} R \\ 0 \end{bmatrix}, \\ C_y &= \begin{bmatrix} I_3 & 0 \end{bmatrix}, & D_{yw} &= 0, & D_{yu} &= 0. \end{aligned} \tag{36}$$

Here $\tilde{x}, z, w \in \mathbb{R}^6$ and $u, q \in \mathbb{R}^3$, so $G \in \mathbb{R}^{15\times 15}$. The actuator frame enters only through $u = Pu_{\mathrm{act}}$ and $q_{\mathrm{act}} = P^\top q$ in (1).

Substituting $w = \Delta(Y)z$ into the z equation of (35) gives the algebraic loop equation $(I_6 - D_{zw}\Delta(Y))z = C_z x + D_{zu}u$. Written out,

$$\begin{bmatrix} I_3 + YRM_1 & YRM_2 \\ -YI_3 & I_3 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \begin{bmatrix} R(u - Kq - C\dot{q}) \\ 0 \end{bmatrix}.$$

The second block row gives $a_1 = Y\ddot{q}$, and the first then gives $M(Y)\ddot{q} = u - C\dot{q} - Kq$, that is (2), after multiplication by M_0. The condition of [16, Def. 1], that $I_6 - D_{zw}\Delta(Y)$ is nonsingular, is therefore the condition that $M(Y)$ is nonsingular: the Schur complement of the lower identity block gives

$$\begin{aligned} \det\big(I_6 - D_{zw}\Delta(Y)\big) &= \det\big(I_3 + YRM_1 + Y^2RM_2\big) \\ &= \det M(Y)/\det M_0. \end{aligned}$$

Since $M(Y)$ is nonsingular, the loop is solved in closed form, $\ddot{q} = M(Y)^{-1}(u - C\dot{q} - Kq)$ with

$$M(Y)^{-1} = \frac{N_0 + YN_1 + Y^2N_2}{d(Y)}, \qquad d(Y) = \det M(Y),$$

where $N_0 + YN_1 + Y^2N_2 = \mathrm{adj}\, M(Y)$ and $d(Y)$ are both quadratic in Y. The implementation evaluates both polynomials with Horner's scheme.

B. Identifiable combinations

For $m_h > 0$, the ten distinct entries of M_0, C and K determine θ_{base} of (11) uniquely (Section II-C):

$$\begin{aligned} &m_h = M_{0,33}, && d = -M_{0,23}/m_h, && m_\Sigma = M_{0,11} - m_h, \\ &m_\Delta = 2M_{0,12}/L_b, && J_{\mathrm{eff}} = M_{0,22} - m_h d^2, && k_{b,\Sigma} = K_{22}, \\ &c_{g1} = \tfrac{1}{2}C_{11} + C_{12}/L_b, && c_{g2} = \tfrac{1}{2}C_{11} - C_{12}/L_b, && c_y = C_{33}, \\ &c_{b,\Sigma} = C_{22} - \tfrac{L_b^2}{4}C_{11}. \end{aligned}$$

TABLE III: Nominal baseline parameters.

Parameter	Value	Parameter	Value
m_1	10.2 kg	m_2	10.7 kg
m_b	22.8 kg	m_h	10.1 kg
J_b	1.00 kg m^2	J_h	0.05 kg m^2
c_{g1}	14.5 N s/m	c_{g2}	20.3 N s/m
c_y	10.0 N s/m	d	0.10 m
c_{b1}	9.0 N m s/rad	c_{b2}	9.0 N m s/rad
k_{b1}	1987.5 N m/rad	k_{b2}	1987.5 N m/rad
L_b	0.725 m		

Conversely, (3) and (4) give every entry as a function of θ_{base}, so the map is one-to-one.

C. Nominal physical parameters

Table III lists the nominal values used by the baseline. They are taken from the ETEL gantry reported by García-Herreros et al. [3, Table I].

D. Positive definiteness of the inertia matrix

With $a = L_b m_\Delta/2$, elimination of the m_h block and completion of the square give

$$\begin{aligned} \frac{\det M(Y)}{m_h} &= (m_\Sigma + m_h)(J_{\mathrm{eff}} + m_h Y^2) - (a - m_h Y)^2 \\ &= m_\Sigma m_h \left(Y + \frac{a}{m_\Sigma}\right)^2 \\ &\quad + (m_\Sigma + m_h)\left(J_{\mathrm{eff}} - \frac{a^2}{m_\Sigma}\right). \end{aligned}$$

The 2×2 Schur complement has positive leading entry $m_\Sigma + m_h$. Sylvester's criterion therefore reduces positive definiteness to positivity of the completed-square expression for every Y. Hence, for positive m_h, m_Σ, and J_{eff}, $M(Y) \succ 0$ for all $Y \in \mathbb{R}$ if and only if (12) holds. For the physical parameterization, the difference between the two sides is

$$\begin{aligned} m_\Sigma J_{\mathrm{eff}} - \frac{L_b^2}{4}m_\Delta^2 &= m_\Sigma(J_b + J_h) \\ &\quad + \frac{L_b^2}{4}\left[4m_1m_2 + m_b(m_1 + m_2)\right], \end{aligned}$$

which is strictly positive for positive component masses and yaw inertias.

In (24), let $b(\theta_{\mathrm{base}}) = \rho(2/L_b)\sqrt{m_\Sigma J_{\mathrm{eff}}}$. The offset and scale are

$$r_0 = \frac{m_{\Delta,0}}{b(\theta_{\mathrm{base},0})}, \quad \eta_0 = \operatorname{atanh}(r_0), \quad s = \frac{m_{\Delta,0}}{b(\theta_{\mathrm{base},0})(1 - r_0^2)}.$$

Here η_0 makes $\xi_\Delta = 0$ reproduce the initial combination, and s gives $\partial m_\Delta/\partial \xi_\Delta|_0 = m_{\Delta,0}$, matching the local relative-change scaling of the logarithmic coordinates. The construction requires $m_{\Delta,0} \neq 0$; a start near zero makes s small and the coordinate numerically inert. The exact inverse used for reporting is

$$\xi_\Delta = \frac{\operatorname{atanh}(m_\Delta/b(\theta_{\mathrm{base}})) - \eta_0}{s}.$$